

Workshop Registration Service Architecture

Overview

The Workshop Registration Service is a full-stack web application designed for internal staff to manage community training center workshops and attendee registrations. It provides a robust, real-time dashboard for managing workshops, handling registrations, and viewing system audits.

Technology Stack

- **Framework:** Next.js 15 (React 19) with App Router
- **Styling:** Tailwind CSS (via PostCSS/Pico)
- **Database:** PostgreSQL (Neon Serverless Postgres / Local Postgres)
- **Authentication:** JWT-based session tokens using `jose`, injected securely on the client-side to overcome edge-middleware restrictions.
- **Data Access:** `pg` node-postgres library, wrapping raw SQL with optimized connection pooling.

Key Features & Design Patterns

Role-Based Access Control (RBAC)

The application defines three strictly enforced roles:

- **Admin:** Has full access to the system, including a dedicated `/admin` dashboard to view system-wide audit logs.
- **Manager:** Can view the main dashboard, create new workshops, edit existing workshops, and manage attendee registrations (register/cancel).
- **Staff:** Can view the main dashboard and manage attendee registrations, but cannot create or edit workshops.

Middleware and layout-level session checks ensure that users cannot access routes they are unauthorized for.

Server Actions & Data Mutations

All data mutations (login, registering attendees, cancelling registrations, creating workshops) are handled securely via React Server Actions. This eliminates the need for standalone API routes while ensuring server-side validation and immediate caching invalidation via `revalidatePath`.

Session Management & The Edge Runtime

A major architectural hurdle in Next.js 15 Server Actions on Vercel Edge involves the stripping of cookies during specific redirects. To overcome this, the application generates a signed JWT session payload on the server, passes the token back in the payload body, and leverages a client-side `document.cookie` assignment followed by `window.location.href` to trigger a hard refresh. This guarantees that session cookies are persisted reliably across server components.

Audit Trails & Data Immutability

- **Registration History:** Attendee registrations are never deleted. Instead, their status is toggled (e.g., `active`, `waitlisted`, `cancelled`), and every action is recorded in the `registration_history` table, tying the action directly to the staff member who performed it.
- **System Audit Logs:** Broad system actions, such as the creation or modification of a workshop, are logged in the `system_audit_logs` table. Admins can review these logs to trace actions back to specific managers.

Database Schema Highlights

- `users`: Stores staff credentials and roles.
- `workshops`: Stores workshop details (capacity, instructor, schedule, status).
- `workshop_types`: Normalizes workshop categories and names.
- `registrations`: Associates attendees with workshops. Uses an `active`, `cancelled`, or `waitlisted` status.
- `registration_history` / `system_audit_logs`: Immutable append-only tables for auditing.

Handling Concurrency & Race Conditions

The Problem: The Last Spot Race Condition

When managing workshop registrations, a classic race condition occurs if multiple staff members attempt to register an attendee for the last remaining spot at the exact same time.

In a naive implementation, both requests would simultaneously query the database, see that there is 1 spot left (e.g., 29 out of 30 registered), and proceed to insert the registration. This results in an overbooked workshop (31/30).

The Solution: Explicit Row Locking (FOR UPDATE)

To serialize these operations and guarantee data consistency without overly restrictive table-level locks, we employ PostgreSQL's explicit row-level locking using the `FOR UPDATE` clause during our transaction block.

How it works:

When `registerAttendee` is invoked, the entire operation is wrapped in a dedicated SQL transaction:

1. Locking the Workshop Row:

```
SELECT capacity FROM workshops WHERE id = $1 FOR UPDATE
```

By adding `FOR UPDATE` to the `SELECT` query, we lock the specific workshop row. If multiple staff members try to register an attendee simultaneously, PostgreSQL forces the second query to **wait** until the first transaction either `COMMITs` or `ROLLBACKs`.

2. Evaluating Capacity Safely:

Because the first transaction holds an exclusive lock on the row, it safely counts the current active registrations:

```
SELECT COUNT(*) as count FROM registrations WHERE workshop_id = $1 AND status = 'active'
```

It then evaluates if `activeCount >= capacity`. If full, it sets the status to `waitlisted`. If not, it sets it to `active`.

3. Inserting & Committing:

The registration is inserted. Once the transaction completes (commits), the row lock is released.

4. The Next Transaction Unlocks:

The second concurrent transaction is now unblocked. It executes its own capacity count. Because the first transaction committed successfully, the count now reflects the new registration. The second transaction correctly evaluates that the workshop is now full, and seamlessly routes the second attendee to the `waitlist`.

Expanding to Cancellations & Waitlist Promotions

We use the exact same lock strategy when cancelling a registration.

If an attendee is cancelled, the spot becomes available. If there is a waitlist, the first waitlisted person should be automatically promoted.

To prevent two concurrent cancellations from accidentally promoting the same waitlisted user:

1. We lock the workshop row (`SELECT id FROM workshops WHERE id = $1 FOR UPDATE`).
2. We update the active registration to `cancelled`.
3. We select the oldest waitlisted user **with a lock** (`... FOR UPDATE LIMIT 1`).
4. We update the waitlisted user to `active` and append an audit trail.
5. The transaction commits and releases the locks.

Conclusion

By strategically applying PostgreSQL's `FOR UPDATE` within transaction boundaries (`BEGIN ... COMMIT`), we completely eliminate race conditions and enforce strict capacity limits without impacting the performance of read-only queries (like the dashboard view).